

GCBF+ 分散式 CBF 基线说明（当前项目实现）

1. 目标与范围
2. 状态、动力学与参考控制
 - 2.1 智能体状态
 - 2.2 名义控制 u_{ref}
3. QP 结构式
4. CLF（软约束）
5. 分散式 CBF 约束
 - 5.1 Agent-Agent 安全函数
 - 5.2 Agent-Obstacle 安全函数
6. 局部分布式感知与“按需加约束”
7. 基线算法流程
8. 关键配置项
9. 运行与输出
 - 9.1 默认场景
 - 9.2 更复杂场景
 - 9.3 输出目录
10. 符号表

GCBF+ 分散式 CBF 基线说明（当前项目实现）

1. 目标与范围

该基线是在当前仓库中实现的**非学习型**安全控制器：不训练策略网络，只在每个时刻对每个智能体求解一个分布式 CBF-QP，完成 2D 多无人机到达任务中的避障与避碰。

对应代码：

- 控制器：[hmarl_cbf/baselines/distributed_cbf.py](#)
- 约束构造：[hmarl_cbf/control/constraint_builder.py](#)
- 运行脚本：[hmarl_cbf/train/run_distributed_cbf_baseline.py](#)
- 默认配置：[configs/hmarl_cbf/baseline_distributed_cbf.yaml](#)

2. 状态、动力学与参考控制

2.1 智能体状态

对第 i 个智能体：

- 位置： $p_i \in \mathbb{R}^2$
- 速度： $v_i \in \mathbb{R}^2$
- 控制输入： $u_i \in \mathbb{R}^2$ (加速度)

双积分器形式：

$$\dot{p}_i = v_i, \quad \dot{v}_i = u_i.$$

2.2 名义控制 u_{ref}

基线中的 u_{ref} 由目标跟踪速度误差得到：

$$v_i^{\text{des}} = s_i^{\text{des}} \hat{g}_i, \quad u_{\text{ref},i} = k_v (v_i^{\text{des}} - v_i),$$

其中：

- $\hat{g}_i = \frac{g_i - p_i}{\|g_i - p_i\|}$ (若距离极小则回退到速度方向/默认方向)；
- s_i^{des} 由 `ref_speed` 与 `slow_radius` 缩放；
- k_v 对应配置 `speed_kp`。

实现位置： `DistributedCBFBaselineController._goal_tracking_u_ref()`。

3. QP 结构式

每个智能体、每个时间步求解：

$$\begin{aligned} \min_{u_i, \delta_i} \quad & \frac{1}{2} u_i^\top \text{diag}(r_i) u_i + f_i^\top u_i + w_{\text{clf}} \delta_i \\ \text{s.t.} \quad & A_{\text{cbf},i} u_i \leq b_{\text{cbf},i}, \\ & A_{\text{clf},i} u_i \leq b_{\text{clf},i} + \delta_i, \\ & u_{\min} \leq u_i \leq u_{\max}, \quad \delta_i \geq 0. \end{aligned}$$

其中：

- 若未显式提供 f_i ，代码取 $f_i = -\text{diag}(r_i) u_{\text{ref},i}$ ，对应“尽量贴近 u_{ref} ”的二次型；
- δ_i 仅用于 CLF 软约束，CBF 约束为硬约束；
- 若求解失败，会走 `deterministic fallback (stub_clipped)`，将动作回退为裁剪后的无约束最优近似。

实现位置:

- 构造 QPProblem: [hmarl_cbf/control/constraint_builder.py](#)
- 求解器: [hmarl_cbf/control/qp_solver.py](#)

4. CLF (软约束)

速度跟踪型 CLF:

$$V_i = \frac{1}{2} \| v_i - v_i^{\text{des}} \|^2,$$

并约束

$$\dot{V}_i \leq -k_{\text{clf}} V_i + \delta_i.$$

在双积分器下, $\dot{V}_i = (v_i - v_i^{\text{des}})^\top u_i$ (忽略 $\dot{v}_i^{\text{des}}$) , 故线性不等式可写为:

$$A_{\text{clf},i} = (v_i - v_i^{\text{des}})^\top, \quad b_{\text{clf},i} = -k_{\text{clf}} V_i.$$

5. 分散式 CBF 约束

基线支持三种模式 (baseline.cbf_mode) :

- distributed_ecbf
- distributed_gcbfplus
- distributed_hocbf54 (默认, 贴近 GCBF+ 文中式(54)风格)

5.1 Agent-Agent 安全函数

设相对量:

$$p_{ij} = p_i - p_j, \quad v_{ij} = v_i - v_j, \quad d_{ij} = \| p_{ij} \|,$$

安全距离 d_{safe} (代码中来自 `d_min_agent`) 。

a) distributed_hocbf54

采用 reciprocal 形式:

$$h_{ij} = \sqrt{4u_{\max}(d_{ij} - d_{\text{safe}})} + n_{ij}^\top v_{ij}, \quad n_{ij} = \frac{p_{ij}}{\| p_{ij} \|}.$$

代码将其线性化为局部仿射约束

$$A_{ij} u_i \leq c_{ij} + k_1 \dot{h}_{ij}^{\text{term}} + k_0 h_{ij}^{\text{term}},$$

其中 share （责任均分）并入 h 与 $\dot{h}$ 项。

b) distributed_gcbfplus

二阶链式：

$$h_0 = \|p_{ij}\|^2 - d_{\text{safe}}^2, \quad h_1 = \dot{h}_0 + \alpha_0 h_0, \\ -L_g h_1 u_i \leq \rho (L_f h_1 + \alpha_1 h_1),$$

其中 ρ 是责任分配系数（ cbf_share_agent ）。

c) distributed_ecbf

等价于 ECBF 风格线性化：

$$\ddot{h} + k_1 \dot{h} + k_0 h \geq 0, \quad h = \|p_{ij}\|^2 - d_{\text{safe}}^2,$$

并整理为对 u_i 的线性不等式。

5.2 Agent-Obstacle 安全函数

对圆障碍中心 o_m 、半径 r_m ，令

$$p_{im} = p_i - o_m, \quad d_{\text{safe},m} = r_m + d_{\text{safe}}^{\text{obs}}$$

采用与上面同构的三种 CBF 形式，仅把相对对象替换为障碍物，责任系数用 cbf_share_obs 。

6. 局部分布式感知与“按需加约束”

控制器并不对所有实体加约束，而是先做本地过滤：

- 邻居过滤：距离 $\leq \text{neighbor_radius}$
- 障碍过滤：障碍表面距离 $\leq \text{obstacle_range}$

只有被感知到的邻居/障碍才写入当前时刻 $A_{\text{cbf}}, b_{\text{cbf}}$ 。这就是“仅对感知到的对象引入 CBF”。

7. 基线算法流程

每个 episode：

1. 环境 reset，采样初始智能体与障碍布局。
2. 每个仿真步，对每个智能体：
 - 计算 u_{ref} ；

- 过滤本地邻居/障碍;
 - 构造 CBF/CLF/输入边界约束;
 - 求解 QP 得到 u_i 。
3. 聚合全部动作推进环境一步。
 4. 记录 `safe_reach_count`、`success_round`、`episode_return_mean`、`qp_feasible_rate` 等指标。
 5. 可选导出 GIF/PNG 轨迹可视化。

8. 关键配置项

- `safety.d_min_agent`: 机间安全距离
- `safety.d_safe_obs`: 障碍附加安全距离
- `baseline.cbf_mode`: CBF 公式分支
- `baseline.cbf_share_agent` / `cbf_share_obs`: 责任分配比例
- `baseline.neighbor_radius` / `obstacle_range`: 感知半径
- `baseline.ref_speed` / `slow_radius` / `speed_kp`: 名义速度跟踪
- `baseline.r_diag` / `w_clf`: QP 目标权重
- `baseline.use_input_bounds`: 是否启用输入限幅

9. 运行与输出

9.1 默认场景

```
python -m hmarl_cbf.train.run_distributed_cbf_baseline \
  --config configs/hmarl_cbf/baseline_distributed_cbf.yaml \
  --run-name gcbfplus_distributed_baseline \
  --episodes 20 \
  --render-gif \
  --fps 8
```

9.2 更复杂场景

```
python -m hmarl_cbf.train.run_distributed_cbf_baseline \
  --config configs/hmarl_cbf/baseline_distributed_cbf_complex.yaml \
  --run-name gcbfplus_baseline_complex_n8_o8 \
  --episodes 30 \
  --render-gif \
  --fps 8
```

9.3 输出目录

- `artifacts/hmarl_cbf_baseline/<run-name>/episode_results.csv`
- `artifacts/hmarl_cbf_baseline/<run-name>/summary.json`
- `artifacts/hmarl_cbf_baseline/<run-name>/media/episode_XXX.gif`

10. 符号表

- p_i, v_i, u_i : 第 i 个智能体的位置、速度、控制输入
- g_i : 目标点
- $u_{\text{ref}, i}$: 名义参考输入
- r_i : QP 二次项对角权重
- f_i : QP 一次项
- δ_i : CLF 松弛变量
- $A_{\text{cbf}}, b_{\text{cbf}}$: CBF 线性不等式矩阵与向量
- $A_{\text{clf}}, b_{\text{clf}}$: CLF 线性不等式矩阵与向量
- k_0, k_1 : CBF 增益
- α_0, α_1 : distributed_gcbfplus 模式下组合增益
- u_{max} : 式(54)型 CBF 中使用的最大控制幅值参数